MARVEL STRATEGO

Guía para la defensa del Proyecto 1 · Programación II

Qué trae esta guía

1. Cómo se ejecuta el programa y el orden de las ventanas
2. Para qué sirve cada clase
3. Cómo se guardan los datos
4. Registro, login y las excepciones
5. Cómo funciona el tablero por dentro
6. Cómo funciona una partida paso a paso
7. Dónde está cada regla del juego
8. Dónde está cada requisito del PDF (REQ-01 a REQ-07)
9. Mi Perfil, Ranking y Batallas
10. Preguntas que te pueden hacer y cómo responderlas

Todas las clases están en el paquete `marvelstrategogame`, dentro de `MarvelStrategoGame/src/marvelstrategogame`. Las imágenes están en la carpeta `imagenes` de ese mismo paquete. Para encontrar un método rápido en NetBeans, abre la clase y busca el nombre con Ctrl+F.

1. Cómo se ejecuta y el orden de las ventanas

El programa empieza en el `main` de **MarvelStrategoGame**, que abre el Menú Inicial. Todo se maneja con ventanas de Java Swing (`JFrame`), nada por consola, como pide el PDF.

```
MarvelStrategoGame (Menú Inicial)  ── EXIT_ON_CLOSE
   ├── LOG IN ──────────► LoginFrame ──(login correcto)──► MenuPrincipal
   │                         └── REGRESAR / X ──► vuelve al Menú Inicial
   └── CREAR PLAYER ──► CrearPlayerFrame
                             └── GUARDAR / CANCELAR / X ──► vuelve al Menú Inicial

MenuPrincipal (barra de menú)
   ├── STRATEGO > Partida Nueva ──► TableroGameFrame (la partida)
   ├── MI PERFIL > Log de Mis Últimos Juegos ──► VentanaInformacion
   │             > Cambiar mi Password / Eliminar mi Cuenta (cuadros de diálogo)
   ├── UNIVERSO MARVEL > Ranking / Batallas ──► VentanaInformacion
   └── CERRAR SESIÓN ──► confirma y vuelve al Menú Inicial
```

- **Regla del PDF:** el Menú Inicial usa `EXIT_ON_CLOSE` y todas las demás ventanas usan `DISPOSE_ON_CLOSE`. Lo puedes ver en el constructor de cada ventana, con `setDefaultCloseOperation(...)`.
- Cuando se abre una ventana, la anterior se cierra con `this.dispose()`. Para regresar, cada ventana tiene un `WindowListener` con el método `windowClosed` que vuelve a abrir la ventana de antes. Por ejemplo, al cerrar la partida o Ranking se abre otra vez el `MenuPrincipal`.

2. Para qué sirve cada clase

Clase	Tipo	Para qué sirve
`MarvelStrategoGame`	JFrame + main	Menú Inicial con los botones LOG IN y CREAR PLAYER.
`LoginFrame`	JFrame	Pide usuario y password, con la opción "Mostrar password". Si están bien, abre el Menú Principal.
`CrearPlayerFrame`	JFrame	Registra un jugador nuevo con sus validaciones.
`MenuPrincipal`	JFrame	Barra de menú con todas las opciones y el saludo al jugador.
`VentanaInformacion`	JFrame	Ventana de texto que reutilizan Mi Perfil, Ranking y Batallas.
`TableroGameFrame`	JFrame	La pantalla de la partida: tablero, turno, RETIRO, piezas vencidas y mensajes.
`Tablero`	Clase lógica	La matriz 10x10 de piezas, la formación inicial y las reglas de movimiento.
`ControladorJuego`	Clase lógica	Guarda de quién es el turno y resuelve los combates.
`GestorJugadores`	Clase lógica	Reglas de los jugadores: registro, login, cambiar y eliminar cuenta, ranking y conteo de victorias.

`DatosGlobales`	Datos (static)	Los "arreglos" del sistema: lista de jugadores, jugador actual, usuarios históricos y ganadores.
`Jugador`	Clase de datos	username, password, puntos, partidas con Héroes y con Villanos, y sus logs.
`Pieza`	**Clase abstracta**	Lo que todas las piezas tienen: nombre, rango, equipo y si está revelada.
`PiezaNormal`	Clase concreta (padre)	Pieza que se mueve 1 casilla y pelea por rango.
`PiezaAgente`	Hija de PiezaNormal	Rango 1 (Black Widow): si ataca a un rango 10, lo vence.
`PiezaMistica`	Hija de PiezaNormal	Rango 2: se mueve varias casillas en línea recta.
`PiezaExplorador`	Hija de PiezaNormal	Rango 3: es la única que destruye bombas.
`Bomba`	Hija de Pieza	Nova Blast o Pumpkin Bomb: no se mueve y destruye a quien la ataca.
`Tierra`	Hija de Pieza	No se mueve; si la capturan, se gana la partida.
`Movible`, `Atacable`	**Interfaces**	Contratos para moverse y para recibir ataques.
`Equipo`, `ResultadoCombate`	**Enums**	HEROES/VILLANOS y los resultados posibles de un combate.
4 clases `...Exception`	**Excepciones propias**	UsuarioDuplicado, PasswordInvalido, CredencialesInvalidas y MovimientoInvalido.
`CeldaTablero`	JButton	Un botón de casilla que dibuja la imagen del personaje y su rango.
`PanelImagen`	JPanel	Panel que dibuja una imagen de fondo (las portadas y el tablero).
`Imagenes`, `Estilo`	Ayudantes	Cargar imágenes y crear los botones, textos y colores iguales en todas las ventanas.

3. Cómo se guardan los datos

El PDF dice que al crear un jugador se genera "un nuevo registro en arreglos". Eso está en **DatosGlobales**:

```
public static ArrayList<Jugador> listaJugadores = new ArrayList<>();   // jugadores activos
public static Jugador jugadorActual;                                    // el que inició sesión
public static int usuariosHistoricos = 0;                               // creados en total (nunca
baja)
public static ArrayList<Equipo> ganadoresPartidas = new ArrayList<>();  // quién ganó cada partida
```

- Son `static` para que todas las ventanas usen la misma lista.
- Cada **Jugador** tiene `username`, `password`, `puntos`, `partidasHeroes`, `partidasVillanos` y una lista `logs` con su historial. Al crearlo, los puntos y los contadores empiezan en 0 dentro del constructor, como pide el PDF.
- Los datos viven en memoria: al cerrar el programa se borran. El PDF no pide guardarlos en archivos.

4. Registro, login y las excepciones

Las ventanas solo leen lo que se escribe. Las **reglas** están en **GestorJugadores**, y cuando algo está mal, el método **lanza (throw)** una excepción propia. La ventana la **atrapa (catch)** y muestra el mensaje, así el programa nunca se cae.

Situación	Dónde se lanza (throw)	Dónde se atrapa (catch)
Password que no tiene exactamente 5 caracteres o no es alfanumérico	`GestorJugadores.validarPassword`	`CrearPlayerFrame` y `MenuPrincipal.cambiarPassword`
Username que ya existe (sin importar mayúsculas ni espacios)	`GestorJugadores.registrar`	`CrearPlayerFrame`
Usuario o password incorrectos	`GestorJugadores.iniciarSesion`, `cambiarPassword` y `eliminarCuenta`	`LoginFrame` y `MenuPrincipal`
Mover a una casilla no permitida	`Tablero.moverPieza` y `Tablero.validarAtaque`	`TableroGameFrame.manejarClickCasilla`

Ejemplo del registro en **CrearPlayerFrame**:

```
try {
    GestorJugadores.registrar(user, pass);
    JOptionPane.showMessageDialog(this, "¡Jugador creado con éxito!");
    this.dispose();
} catch (PasswordInvalidoException | UsuarioDuplicadoException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage());
}
```

Mostrar password: en `Estilo.crearMostrarPassword` hay una casilla (`JCheckBox`). Si se marca, se usa `setEchoChar((char) 0)` para que se vean las letras; si se desmarca, se vuelven a poner los puntitos.

5. Cómo funciona el tablero por dentro (clase Tablero)

5.1 La matriz

El tablero es una matriz `Pieza[10][10]` llamada `matriz`. Cada casilla guarda una `Pieza` o `null` si está vacía. Como la matriz es de tipo `Pieza` (la clase padre), en ella caben piezas normales, bombas y Tierras al mismo tiempo. Eso es **polimorfismo**.

- **Filas:** 0 a 3 son de los Villanos y 6 a 9 de los Héroes. Las filas 4 y 5 empiezan vacías.
- **Zonas prohibidas:** el método `esZonaProhibida(fila, col)` devuelve true en las filas 4-5 y las columnas 2, 3, 6 y 7. Son los dos cuadros 2x2 del centro, que en la imagen del tablero son las esferas amarilla y morada.

5.2 Formación inicial (las 5 reglas del PDF)

El método `colocarPiezasIniciales()` llama a `colocarEquipo(...)` una vez por equipo. Los nombres de los 40 personajes de cada equipo están en los arreglos `HEROES` y `VILLANOS`: la primera línea es rango 1, la segunda rango 2, y así. Los pasos son:

1. **Tierra:** se pone al azar en la última fila (0 o 9), nunca en la columna 0 ni en la 9 (`1 + azar.nextInt(8)`). Alrededor de ella van 3 bombas: a la izquierda, a la derecha y enfrente.
2. **Bombas restantes:** las otras 3 van al azar en las últimas 2 filas del equipo.
3. **Rango 2:** las 8 piezas de rango 2 van al azar en las filas 2-3 (Villanos) o 6-7 (Héroes).
4. **Primera fila:** se llena lo que quede libre de la fila 3 o 6.
5. **El resto:** las piezas que sobran llenan los espacios vacíos.

Para colocar piezas al azar se usa el método recursivo `colocarAleatorio` (ver sección 8, REQ-05). El método `crearPieza(nombre, rango, equipo)` decide con un `switch` qué clase crear: rango 1 da `PiezaAgente`, rango 2 `PiezaMistica`, rango 3 `PiezaExplorador` y cualquier otro `PiezaNormal`.

5.3 Mover una pieza: moverPieza(fOri, cOri, fDes, cDes)

Revisa las reglas en orden. Si una falla, lanza `MovimientoInvalidoException` y la pieza no se mueve:

1. ¿El destino es una zona prohibida? Entonces da error.
2. ¿El destino está ocupado? Entonces da error (para pelear se ataca, no se mueve).
3. ¿La pieza se puede mover así? Esto lo responde `esMovimientoValido` de la pieza real, a través de la interfaz `Movible`.
4. ¿El camino está libre? Esto lo revisa `caminoDespejado` (recursivo), y le importa sobre todo al rango 2.
5. Si todo está bien, mueve la pieza en la matriz. Si avanzó más de 1 casilla, era un rango 2 y queda **revelada**, como dice el PDF.

`validarAtaque` revisa que el ataque sea a una casilla **de al lado** (sin diagonales), porque el PDF dice "puede atacar a una pieza adyacente enemiga".

5.4 ¿Una pieza o un equipo pueden jugar?

- `piezaPuedeActuar(fila, col)` revisa las 4 casillas de alrededor (arriba, abajo, izquierda y derecha). La pieza puede jugar si alguna está dentro del tablero, no es zona prohibida y está vacía o tiene un enemigo. Bombas y Tierra siempre dan false, porque nunca se mueven.

- `tieneMovimientos(equipo)` recorre todo el tablero (recursivo) buscando una pieza del equipo que pueda jugar. Si no encuentra ninguna, ese equipo pierde.

6. Cómo funciona una partida (clase TableroGameFrame)

6.1 Empezar

En **MenuPrincipal.mostrarPartidaNueva()** el jugador elige su **oponente** y su **equipo**, y el oponente se queda con el otro. Si solo hay 1 jugador registrado, la opción Partida Nueva sale deshabilitada (`setEnabled(false)`), porque el PDF dice que "no está disponible". Después se crea `new TableroGameFrame(jugadorHeroes, jugadorVillanos)`, que coloca las piezas y dibuja el tablero.

6.2 Lo que se ve en pantalla

- **Arriba:** el turno ("Turno del equipo HÉROES de ana") y el botón **RETIRO**.
- **Centro:** 100 botones `CeldaTablero` en un `GridLayout(10, 10)` encima de la imagen del tablero. Cada botón sabe su fila y columna y llama a `manejarClickCasilla(fila, col)`.
- **A los lados:** las piezas vencidas de cada equipo (azul Héroes, rojo Villanos). Son listas `JList` con un `DefaultListModel`, y al eliminar una pieza se agrega con `registrarVencida`. El PDF pide que se vean todo el tiempo.
- **Abajo:** una barra con los mensajes (errores en rojo) y el botón **VER MIS PIEZAS**.

6.3 Cómo se juega con 2 clics

1. **Primer clic: seleccionar.** El método `seleccionarPieza` revisa que la casilla tenga una pieza, que sea del equipo en turno y que se pueda mover. Si todo está bien, la pieza queda con borde amarillo.
2. **Segundo clic: mover o atacar.** El método `ejecutarJugada` hace esto:
   - Si el destino está vacío, llama a `tablero.moverPieza(...)`.
   - Si hay un enemigo, llama a `validarAtaque`, revela las dos piezas y le pide a `ControladorJuego.resolverCombate` el resultado. Con un `switch` sobre el enum `ResultadoCombate` decide qué piezas quedan y cuáles se van a vencidas.
   - Si hay un error, la `MovimientoInvalidoException` se atrapa y el mensaje sale en la barra de abajo.
3. **Revisar si terminó.** Si se capturó la Tierra, gana el que atacó. Si el rival no tiene ninguna jugada (`tieneMovimientos`), pierde. Si no pasó nada de eso, se cambia el turno.

6.4 Fichas ocultas y cambio de turno

El PDF dice que las fichas se colocan con la identidad oculta para el adversario. El método `piezaVisible(pieza)` decide qué se ve:

- Una pieza **revelada** (porque peleó o porque el rango 2 avanzó varias casillas) la ven los dos jugadores.
- Las piezas del equipo en turno las ve ese jugador.
- Las demás salen como un **"?"** del color de su equipo.

Al cambiar de turno (`pasarTurno`), se tapan todas las piezas (`ocultarTodo = true`) y aparece el botón **VER MIS PIEZAS**. Así, mientras se pasan la computadora, nadie ve las piezas del otro. Cuando el siguiente jugador presiona el botón (`verMisPiezas`), se destapan las suyas.

6.5 Fin de la partida, puntos y logs

Todo pasa en `terminarPartida(ganador, textoLog)`:

- Le suma 3 puntos al ganador y 1 partida a cada jugador según su equipo (`partidasHeroes` o `partidasVillanos`).
- Guarda el log en los dos jugadores con `logs.add(0, log)`. La posición 0 hace que quede de primero, y así el historial va del más reciente al más viejo, como pide el PDF.
- Guarda el equipo ganador en `DatosGlobales.ganadoresPartidas`, que se usa en Batallas.
- Muestra el log, cierra la partida y vuelve al Menú Principal.

Los 3 formatos de log del PDF (la fecha se pone con `LocalDateTime.now()`):

Cuándo	Texto que se guarda
Capturar la Tierra	ana usando HÉROES ha SALVADO la TIERRA! Venciendo a beto — 08/11/2026 16:47 (los Villanos dicen CAPTURADO)
Rival sin movimientos	beto usando VILLANOS ha perdido por no tener movimientos válidos disponibles ante ana — FECHA
RETIRO	beto usando VILLANOS ha ganado ya que ana usando HÉROES se ha retirado del juego — FECHA

RETIRO (`retirarse()`): pide confirmación con `showConfirmDialog`. Si se confirma, pierde el equipo en turno y el otro gana 3 puntos.

7. Dónde está cada regla del juego

Regla del PDF	Dónde está
Empiezan siempre los HÉROES	Constructor de `ControladorJuego`: `turnoActual = Equipo.HEROES`
Turnos alternados	`ControladorJuego.cambiarTurno()`
1 casilla, sin diagonal	`PiezaNormal.esMovimientoValido`: la suma de las diferencias de fila y columna debe ser 1
No moverse a zona prohibida ni a casilla propia	`Tablero.moverPieza` y `Tablero.piezaPuedeActuar`
Rango 2 se mueve varias casillas en línea recta	`PiezaMistica.esMovimientoValido` + `Tablero.caminoDespejado`
Rango 2 se revela al moverse varias casillas	Final de `Tablero.moverPieza`
Solo se ataca a una pieza de al lado	`Tablero.validarAtaque`
Al atacar se revelan las dos piezas	`TableroGameFrame.ejecutarJugada` (`setRevelada(true)`)
Gana el rango mayor; si empatan salen las dos	`PiezaNormal.recibirAtaque`
Rango 1 que ataca a rango 10 gana	`PiezaAgente.derrotaAlRango10()` devuelve true; lo usa `PiezaNormal.recibirAtaque`
Bomba destruye a todos menos al rango 3	`Bomba.recibirAtaque` + `PiezaExplorador.puedeDesactivarBombas()`
Bomba y Tierra no se mueven	`esMovimientoValido` devuelve false en `Bomba` y `Tierra`
Capturar la Tierra da la victoria	`Tierra.recibirAtaque` devuelve `VICTORIA_TIERRA`
Sin movimientos = se rinde y pierde	`Tablero.tieneMovimientos`, usado en `ejecutarJugada`
Piezas vencidas visibles todo el tiempo	Listas laterales de `TableroGameFrame`
RETIRO con confirmación y 3 puntos al oponente	`TableroGameFrame.retirarse`

8. Requisitos de Programación Orientada a Objetos

REQ-01 · Herencia sin clase abstracta

Dónde: `PiezaNormal` es una clase **concreta** (se puede hacer `new PiezaNormal(...)`) y es la clase padre de tres clases concretas:

```
public class PiezaAgente     extends PiezaNormal   // rango 1
public class PiezaMistica    extends PiezaNormal   // rango 2
public class PiezaExplorador extends PiezaNormal   // rango 3
```

Las 3 heredan todo de PiezaNormal y cambian solo su parte especial con `@Override`: el Místico cambia cómo se mueve, el Explorador puede desactivar bombas y el Agente vence al rango 10.

REQ-02 · Clase abstracta

Dónde: `public abstract class Pieza`. No tiene sentido crear una "pieza" genérica, solo sus tipos. Tiene los atributos comunes (`nombre`, `rango`, `equipo`, `revelada`) y dos **métodos abstractos**:

```
public abstract boolean esMovimientoValido(int fOri, int cOri, int fDes, int cDes);
public abstract ResultadoCombate recibirAtaque(Pieza atacante);
```

Cada subclase los implementa distinto. `recibirAtaque`: `PiezaNormal` compara rangos, `Bomba` destruye al atacante (menos al rango 3) y `Tierra` da la victoria.

REQ-03 · Interfaces (2)

Interfaz	Método	Quién la implementa
`Movible`	`boolean esMovimientoValido(...)`	`Pieza` (y por herencia todas las piezas)
`Atacable`	`ResultadoCombate recibirAtaque(Pieza atacante)`	`Pieza` (y por herencia todas las piezas)

```
public abstract class Pieza implements Movible, Atacable
```

Las interfaces solo dicen **qué** se hace, no **cómo**: no tienen atributos ni código dentro de los métodos, como pide el PDF.

REQ-04 · Enums (2)

Enum	Valores	Dónde se usa para decidir
`Equipo`	HEROES, VILLANOS	El turno en `ControladorJuego`, de qué equipo es cada pieza, el combo "Tu equipo", los colores y los logs.
`ResultadoCombate`	GANA_ATACANTE, GANA_DEFENSOR, EMPATE, VICTORIA_TIERRA	Lo devuelve `recibirAtaque` y lo usa el `switch (resultado)` de `TableroGameFrame.ejecutarJugada`.

REQ-05 · Recursividad (hay 5, el PDF pide 3)

Función	Problema que resuelve	Caso base
`Tablero.caminoDespejado` (R1)	Revisa casilla por casilla si el camino del rango 2 está libre.	Llegó al destino (true) o encontró algo (false).
`Tablero.tieneMovimientos` (R2)	Recorre el tablero buscando una pieza del equipo que pueda jugar.	Se pasó de la fila 9 sin encontrar (false) o encontró una (true).
`Tablero.colocarAleatorio` (R4)	Pone una pieza al azar en una celda libre y sigue con las demás.	No quedan piezas o no quedan celdas.
`GestorJugadores.insertarOrdenado` (R5)	Mete un jugador en la lista del ranking en su lugar según los puntos.	Llegó al final de la lista o encontró a alguien con menos puntos.

`GestorJugadores.contarVictorias` (R3)	Cuenta cuántas partidas ganó un equipo recorriendo la lista de ganadores.	Se pasó del final de la lista: devuelve 0.

Ejemplo para explicar en la defensa:

```
public static int contarVictorias(ArrayList<Equipo> ganadores, Equipo bando, int posicion) {
    if (posicion == ganadores.size()) {          // CASO BASE: ya no hay más partidas
        return 0;
    }
    int esta = (ganadores.get(posicion) == bando) ? 1 : 0;
    return esta + contarVictorias(ganadores, bando, posicion + 1);   // LLAMADA RECURSIVA
}
```

REQ-06 · Polimorfismo

- **Con herencia:** la matriz `Pieza[10][10]` y las listas `ArrayList<Pieza>` guardan piezas normales, bombas y Tierras mezcladas. Cuando se llama `pieza.esMovimientoValido(...)`, Java ejecuta la versión de la pieza real.
- **Con interfaces:** en `ControladorJuego.resolverCombate` el defensor se guarda en una variable de tipo interfaz, y en `Tablero.moverPieza` pasa lo mismo con `Movible`:

```
public ResultadoCombate resolverCombate(Pieza atacante, Pieza defensor) {
    Atacable objetivo = defensor;               // referencia de tipo interfaz
    return objetivo.recibirAtaque(atacante);    // Java elige Tierra, Bomba o PiezaNormal
}
```

Por eso el combate no necesita ningún `if (defensor instanceof Bomba)`: cada pieza sabe responder a su manera.

REQ-07 · Excepciones propias (hay 4, el PDF pide 3)

Todas hacen `extends Exception`, y cada una se lanza y se atrapa en el lugar correcto (ver la tabla de la sección 4): `UsuarioDuplicadoException`, `PasswordInvalidoException`, `CredencialesInvalidasException` y `MovimientoInvalidoException`.

9. Mi Perfil, Ranking y Batallas (clase MenuPrincipal)

Opción	Método	Qué hace
Log de Mis Últimos Juegos	`mostrarPerfil()`	Muestra username, puntos, partidas con Héroes y con Villanos (sin el password) y el historial del más reciente al más viejo.
Cambiar mi Password	`cambiarPassword()`	Pide el password actual para verificar y el nuevo. `GestorJugadores.cambiarPassword` revisa los dos.
Eliminar mi Cuenta	`eliminarCuenta()`	Pide el password. Si coincide, quita al jugador de `listaJugadores` y regresa al Menú Inicial.
Ranking	`mostrarRanking()`	Jugadores activos de mayor a menor puntaje (ordenados con la recursión `insertarOrdenado`). Formato: `1. ana 9 pts`.
Batallas	`mostrarBatallas()`	Usuarios activos (`listaJugadores.size()`), usuarios históricos (`usuariosHistoricos`, que incluye a los eliminados porque nunca se resta), partidas jugadas (`ganadoresPartidas.size()`) y victorias de cada equipo (`contarVictorias`).
Cerrar Sesión	`cerrarSesion()`	Pide confirmación, cierra el Menú Principal y abre el Menú Inicial.

Mi Perfil, Ranking y Batallas se abren en **VentanaInformacion**, que recibe un título y un texto. Así se reutiliza una sola clase para las tres. Mientras está abierta, el Menú Principal se cierra, y al presionar REGRESAR vuelve.

10. Preguntas que te pueden hacer

¿Por qué Pieza es abstracta?

Porque una "pieza" en general no existe en el juego: siempre es una pieza normal, una bomba o la Tierra. La clase abstracta junta lo que todas tienen y obliga a cada una a decir cómo se mueve y cómo responde a un ataque.

¿Cuál es la diferencia entre clase abstracta e interfaz en tu proyecto?

`Pieza` (abstracta) tiene atributos y métodos con código, como `getNombre()` y `getRutaImagen()`. `Movible` y `Atacable` (interfaces) solo dicen qué métodos debe haber, sin atributos ni código.

¿Dónde se ve el polimorfismo?

En `resolverCombate`: una variable `Atacable` llama a `recibirAtaque` y Java ejecuta la versión de la pieza real. También cuando la matriz de `Pieza` llama a `esMovimientoValido` y cada pieza se mueve a su manera.

¿Cómo sabe el programa que el rango 2 puede moverse varias casillas?

Las piezas de rango 2 se crean como `PiezaMistica` (en `Tablero.crearPieza`), y esa clase sobrescribe `esMovimientoValido` para aceptar cualquier distancia en línea recta. Después `caminoDespejado` revisa que no haya nada en el camino.

¿Cómo se gana?

De 3 formas: capturando la Tierra (`VICTORIA_TIERRA`), cuando el rival ya no tiene movimientos (`tieneMovimientos` da false, y eso incluye que le destruyeron todas sus piezas móviles) o cuando el rival se retira.

¿Cómo se evita que un jugador vea las piezas del otro?

Con `piezaVisible`: solo se ven las piezas propias y las ya reveladas. Al cambiar de turno se tapa todo hasta que el nuevo jugador presiona VER MIS PIEZAS.

¿Qué pasa si un usuario se registra como "Ana" y ya existe "ana"?

`GestorJugadores.buscar` compara con `equalsIgnoreCase` y `trim()`, así que lo toma como el mismo usuario y `registrar` lanza `UsuarioDuplicadoException`.

¿Dónde se guardan los jugadores?

En el `ArrayList<Jugador> listaJugadores` de `DatosGlobales`, que es `static` para que todas las ventanas usen la misma lista. Se guardan en memoria mientras el programa está abierto.

¿Por qué el programa no se cae si se hace una jugada mala?

Porque `Tablero` lanza `MovimientoInvalidoException` y `TableroGameFrame` la atrapa con try-catch, mostrando el mensaje en la barra de abajo.

> **Tip para la defensa:** antes de entrar, abre en NetBeans `Tablero.java`, `TableroGameFrame.java`, `Pieza.java`, `ControladorJuego.java` y `GestorJugadores.java`. En esas 5 clases está casi todo lo que te pueden preguntar. Juega una partida corta antes y prueba un retiro, para que tengas logs en Mi Perfil y datos en Ranking y Batallas.

> **Cosas que cambiaron respecto al PDF (y por qué):**
>
> - **Ranking:** sale como "1. ana 9 pts". Muestra la posición, el usuario y los puntos que pide el PDF, solo escritos distinto.
> - **Password:** además de los 5 caracteres, debe tener letras y números.
> - **Sin modo tutorial:** por indicación del ingeniero, Partida Nueva solo pide oponente y equipo.
> - **Puntos:** cualquier victoria da 3 puntos. El PDF solo lo dice del retiro, y el Ranking necesita puntos para ordenar.